

MACHINE LEARNING ENGINEERING

FASE 5 | AULA 01 -

CONCEITO DE REPRODUTIBILIDADE EM CIÊNCIA DE DADOS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
MERCADO, CASES E TENDÊNCIAS	11
O QUE VOCÊ VIU NESTA AULA?	12
REFERÊNCIAS.....	13

EMSE

O QUE VEM POR AÍ?

Quando pensamos no processo de “colocar um modelo no ar”, saímos do ambiente local, estruturado para pesquisa e experimentação, para outro ambiente onde o modelo é usado por outras pessoas, através de alguma interface de uso, como uma API Rest. E localmente isso funciona muito bem, porque o ambiente de operação (o “ambiente”) é conhecido e alterável.

Mas e quando precisamos que este ambiente seja replicado, seja em um ambiente de operação para consumo de usuários externos (“produto”) ou para colaboração, embarcando outras pessoas no processo?

O processo de produtização de um modelo é, ao mesmo tempo, fascinante e desafiador. Porque, se estamos publicando um modelo de predição para uso de um público definido, precisamos fazer isso com critério e processo bem definidos.

Um dos pilares desse processo é o gerenciamento de dependências, porque serve para definir o ambiente operacional onde todo o processo vai rodar, desde o treinamento automatizado até a disponibilização dos modelos.

Vamos entender porque a reprodutibilidade é importante no processo de produtização dos dados e como isso tem impacto direto na qualidade e no funcionamento do seu pipeline de dados automatizado, do treinamento à publicação do modelo.

HANDS ON

Nesta aula, vamos entender o que é a reprodutibilidade, qual o objetivo dela e porque é importante pensar na reprodutibilidade como um requisito para a qualidade do processo de produtização do modelo. Essa base é importante para que os próximos passos, mais técnicos, sejam claros e entendidos não apenas como comandos, mas como parte de um processo maior e mais amplo.

Vamos também ver qual é o modo “padrão” de controle de dependências no Python (o venv) para entender o porquê deste modo de controle ser precário e não atender às necessidades modernas de criação de processos de dados.

SAIBA MAIS

Vamos falar de Reprodutibilidade

Você deve conhecer bem essa situação: construiu um modelo excelente em um ambiente local de análise dos dados, preparou toda a estrutura, mas viu a execução falhar ao ser migrada para um servidor diferente.

Na engenharia de software tradicional, o clássico "na minha máquina funciona" já representa uma grande dor de cabeça, mas no universo de dados e Machine Learning o problema assume uma proporção muito maior e dependendo da estrutura do projeto, exige uma solução muito mais complexa. Porque nesse cenário, o sucesso não depende exclusivamente da lógica do código escrito, mas de uma equação delicada e inseparável:

- Código.
- Dados.
- E o ambiente exato onde a execução ocorre.

Se você entende este problema, seja por ter passado por ele, ou por imaginar que isso poderia acontecer, já consegue entender que esse é o principal desafio da reprodutibilidade: fazer com que um ambiente seja replicado em outro, para que os processos de Machine Learning funcionem como esperado em qualquer ambiente de execução.

Por conta de seu caráter de experimentação, o ambiente de ciência de dados depende de uma teia profunda e muitas vezes invisível de dependências cruzadas. Isso já é um problema quando estamos trabalhando com aplicações em qualquer linguagem, em que a instalação de uma biblioteca carrega uma quantidade de outras dependências, mas na área de ciências de dados isso tende a ser um complicador maior.

Se pensamos no ambiente Python de exploração de dados, uma simples importação de bibliotecas como pandas ou torch carrega consigo dezenas de outros pacotes, e pode também depender de requisitos específicos de compiladores C para compilação de bibliotecas nativas ao Sistema Operacional e arquitetura do

computador, drivers de hardware (para CPU e GPU) e nuances do próprio sistema operacional.

Então se você roda sua exploração de dados e treinamento de modelos localmente em seu computador que roda Windows com processador de arquitetura amd64, dependendo de como você fez a instalação dos módulos, talvez não seja possível que outra pessoa rodando MacOS com processador Apple Silicon (arquitetura arm64) consiga rodar os treinamentos, dado que podem existir diferenças de bibliotecas e da disponibilidade dessas.

Outro desafio é o controle de versões do Python. Como a linguagem tem um ciclo rápido de desenvolvimento, muitas vezes uma nova versão “major” do Python é disponibilizada e pensamos que, usando a versão mais nova, teremos acesso a melhorias de desempenho e segurança; mas isso também é um problema, dado que as bibliotecas precisam ser, muitas vezes, adaptadas para estas novas versões.

Então se você treina seu modelo com o Python 3.12, pode ser que outra pessoa que usa o Python 3.13 não consiga replicar o funcionamento localmente por conta de o suporte de versão de uma biblioteca ainda não estar disponível para esta nova versão do Python, ou pode ocorrer que alguém usando uma versão anterior do Python também pode enfrentar outros problemas.

Então, vimos que o desafio está ligado diretamente ao ambiente. E, recapitulando, quando falamos de ambiente de execução, falamos de:

- Sistema operacional.
- Versão do Python.
- Versão das bibliotecas usadas.
- Configuração do equipamento usado (arquitetura de CPU, GPU).

Qualquer uma dessas variáveis tem o potencial de causar problema. E quando combinadas, o potencial sobe. E isso não tem a ver com o processo de treinamento de modelos, mas sim com o processo de publicação e produtização dos modelos.

Fica claro o problema que podemos exemplificar com outro exemplo, mais complexo, e com um comportamento que parece inofensivo a princípio: o ambiente de operação não possui um controle rigoroso de versões e é realizada uma

atualização em uma biblioteca matemática que pode alterar a forma como cálculos de ponto flutuante são resolvidos. O grande perigo, nesse caso, é que a aplicação raramente falha com um erro explícito; em vez disso, o modelo sofre uma degradação silenciosa em seus resultados, tornando o diagnóstico extremamente complexo.

Daí que, no contexto da Engenharia de Machine Learning, superar o desafio da reprodutibilidade deixou de ser apenas um capricho técnico, um “nice to have”, para se tornar um pilar obrigatório na construção de soluções confiáveis. E é justamente essa disciplina e previsibilidade na estruturação e gestão de dependências, totalmente relacionada ao processo da produtização de modelos e aplicações de dados, que prepara o caminho para a implementação de práticas maduras de MLOps, permitindo a transição segura da fase de pesquisa para a operação em larga escala.

E para isso, vamos usar algumas das ideias que o DevOps criou e adaptar para a realidade da análise de dados. Por exemplo, podemos pensar em como padronizar uma estrutura que considere que os diferentes ambientes têm que se comportar do mesmo jeito e dominar a gestão desses ambientes, o que pode significar a criação de “cápsulas computacionais” isoladas, garantindo que um pipeline executado hoje gere o exato mesmo resultado se for reexecutado daqui a meses por outro membro da equipe, em outro equipamento, em outro sistema operacional.

Vantagens de garantir o processo de reprodutibilidade

Ficou claro que garantir a reprodutibilidade do ambiente de operação é importante para que o processo funcione de forma adequada. Mas ainda não está claro, de forma explícita, quais são as vantagens de se garantir que o processo funcione da mesma forma em qualquer lugar. Podemos identificar três principais objetivos que a reprodutibilidade de ambientes visa:

- **Rigor Científico e Auditoria:** na ciência de dados, um experimento (como o treinamento ou a inferência de um modelo de Machine Learning) só possui validade real se puder ser recriado com os mesmos resultados, baseados nos mesmos dados, em qualquer ambiente. A reprodutibilidade

garante que a equação Código + Dados + Ambiente = Resultado seja sempre verdadeira. Isso é essencial para auditorias de modelos, validação de hipóteses e rastreamento de mudanças em retreinos constantes.

- **Transição Contínua para Produção (MLOps):** se em Engenharia de Software “comum” falamos sobre o processo de integração contínua, aqui pensamos da mesma forma, mas focando nos diferentes passos de um projeto de Machine Learning. O caminho que começa no ambiente de experimentação (como um Jupyter Notebook) até um servidor de produção exige paridade absoluta, comportamento previsível em todos os passos. Garantir um ambiente reprodutível significa que o mesmo ecossistema de bibliotecas e binários que validou o modelo também fará a inferência em produção, viabilizando pipelines de CI/CD (Integração e Entrega Contínuas) seguros.
- **Colaboração Descentralizada:** em equipes multidisciplinares (times de ciência de dados, engenharia de software, e operação), os códigos e modelos vão transitar entre engenheiros de dados, cientistas de dados e engenheiros de Machine Learning, todos podendo fazer alterações dentro de sua área de responsabilidade. A reprodutibilidade permite que o trabalho seja compartilhado, testado e evoluído de forma assíncrona, garantindo que todos falem a “mesma língua” em termos de ambiente de operação. É importante pensar, nesse momento, que não estamos definindo que todas as pessoas envolvidas no processo têm que ter o mesmo tipo e configuração do equipamento usado; ao contrário, temos que garantir que, dados diferentes ambientes de trabalho, o processo funcione sem sobressaltos e sem a dependência de uma pessoa ou time para que sempre “ajuste” o equipamento das outras pessoas envolvidas no projeto.

E quais problemas a garantia da reprodutibilidade vai resolver?

Se temos as vantagens listadas anteriormente, temos também os pontos de atenção que a reprodutibilidade busca resolver. Aqui, é importante entender que a ideia é mitigar estes problemas; mas dependendo do processo ou da organização

onde o projeto roda, estes problemas não podem ser resolvidos apenas com soluções técnicas.

- **A Síndrome do "Na Minha Máquina Funciona":** elimina a discrepância entre ambientes de desenvolvimento, de testes, de validação e servidores. Evita a frustração de ter um pipeline rodando perfeitamente em um equipamento local, onde tudo roda “muito bem e rápido”, mas quebrando ambiente de produção, na nuvem, devido a versões do sistema operacional, da linguagem de programação disponível ou pacotes conflitantes.
- **O Inferno de Dependências (Dependency Hell):** como falamos anteriormente, em ciência de dados, bibliotecas possuem árvores de dependências muito profundas e sensíveis, que dependem de variáveis que não são facilmente endereçadas ou corrigidas. Usando um processo de isolamento de ambientes evitamos, por exemplo, que a atualização do Pandas em um projeto corrompa funções legadas em outro projeto que depende de uma versão anterior.
- **Regressões Silenciosas (ou drifting inesperado) de Modelos:** se estamos falando sobre dependências, precisamos falar também sobre atualizações de pacotes, seja por necessidade de atualização de segurança ou atualização de versão de bibliotecas e runtime/versão do Python. A ideia é evitar que atualizações não rastreadas em bibliotecas matemáticas (como scikit-learn, PyTorch ou drivers CUDA) alterem, mesmo que sutilmente, a forma como os cálculos são realizados nos bastidores. Sem reprodutibilidade, um modelo pode perder precisão (accuracy) ou apresentar desvios (drift) em produção simplesmente porque o pacote de cálculo matricial foi atualizado, não por uma mudança nos dados, dificultando imensamente o diagnóstico de erros.
- **Onboarding de pessoas no projeto:** algo que os times de Engenharia de Software têm trabalhado, e que os times de Engenharia e Ciências de dados podem se aproveitar, é a prática de criar ambientes reproduzíveis para reduzir (muitas vezes drasticamente) o tempo necessário para que novos membros da equipe configurem o ambiente de trabalho necessário

para começar a entregar resultados e alterações. Um ambiente reprodutível reduz semanas de instalação de dependências e configuração de variáveis de ambiente para a execução de um ou dois comandos de inicialização.

EMEND

MERCADO, CASES E TENDÊNCIAS

Leia o artigo “Machine Learning Operations (MLOps) For Beginners”, de Prasad Mahamulkar:

<https://towardsdatascience.com/machine-learning-operations-mlops-for-beginners-a5686bfe02b2/>

EMEND

O QUE VOCÊ VIU NESTA AULA?

Para o processo de transformação de um modelo de predição em um produto, precisamos garantir que o processo funcione sempre, sem depender de configurações específicas para um sistema operacional ou um tipo de servidor.

O controle de dependências é parte fundamental nesse processo, porque aqui é que garantimos que todas as bibliotecas de todos os programas que fazem parte do nosso pipeline de dados estejam corretas, adequadas ao processo e compatíveis, sem dependências específicas que possam inviabilizar o uso do nosso modelo.

Apresentamos também o pacote venv do Python, que fornece uma primeira tentativa de controle de dependências para projetos, mas que não é adequado para projetos mais complexos, onde precisamos de controle de dependências por ambiente, especificação de versão do Python e criação de ambientes virtuais automaticamente.

REFERÊNCIAS

MAHAMULKAR, P. **Machine Learning Operations (MLOps) For Beginners**. 2024. Disponível em: <https://towardsdatascience.com/machine-learning-operations-mlops-for-beginners-a5686bfe02b2/>. Acesso em: 02 abril 2026.

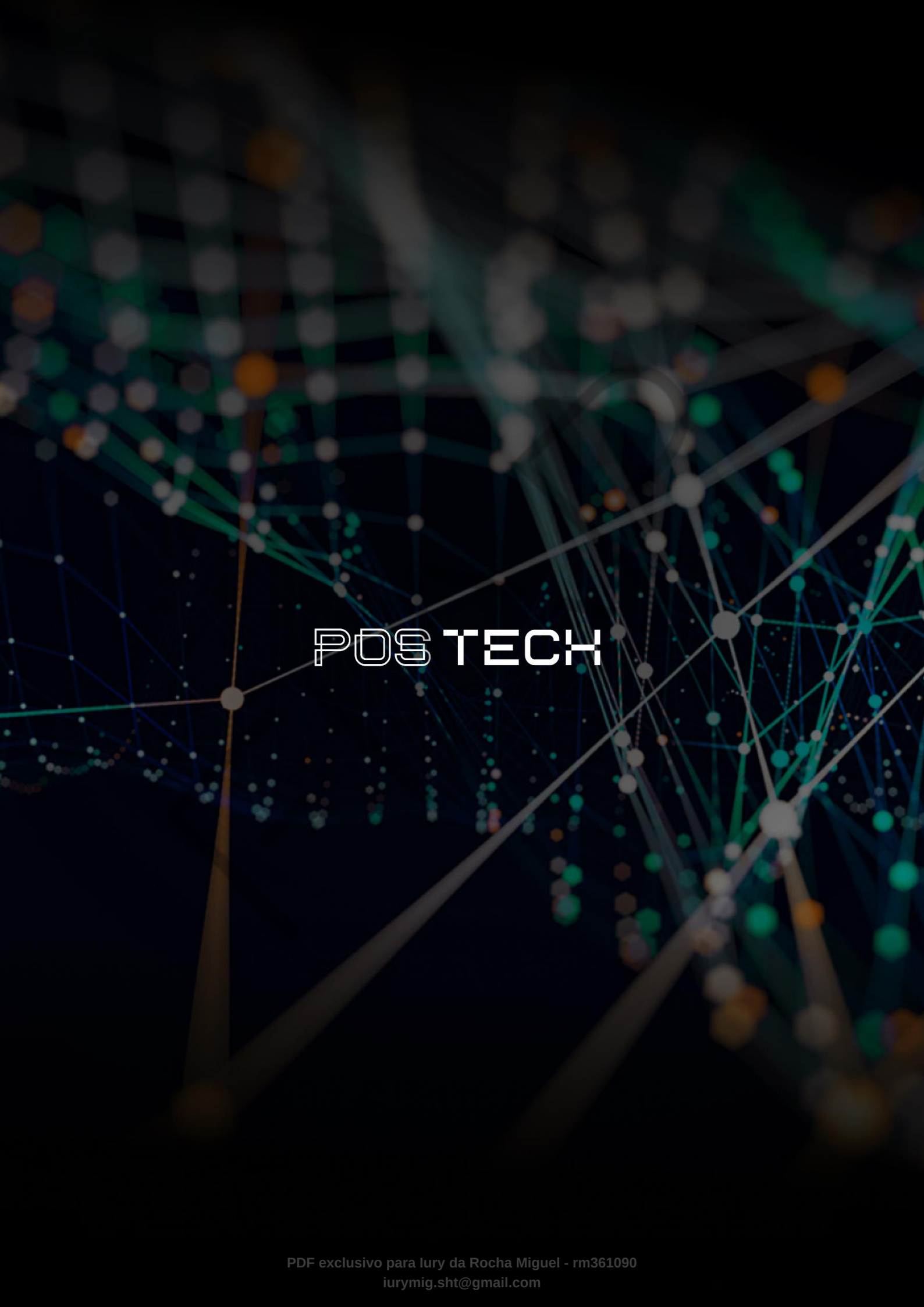
MLOPS. **Machine Learning Operations**. 2026. Disponível em: <https://ml-ops.org/>. Acesso em: 21 abr. 2026. 02 abril 2026.

POTE, S. **Machine Learning in Production**. [s.l.]: BPB Publications, 2023.

PALAVRAS-CHAVE

Reprodutibilidade. Ambiente de execução. Dependências. MLOps. Portabilidade.

EMEND

The background is a dark, abstract network visualization. It features a complex web of interconnected nodes and lines. The nodes are represented by small, semi-transparent circles in various colors, including teal, orange, and grey. The lines connecting them are thin, light-colored threads that create a dense, web-like structure. Some lines are thicker and more prominent, forming a larger geometric framework. The overall effect is one of digital connectivity and data flow.

POSTECH